Chapter 10 - Creating Text Embedding Models

Embedding Models

As we know, textual data as they are cannot be processed. Just as texts, we cannot visualize, process, or do much with them.

As we learned from the previous chapters, we first have to convert this textual data to *numeric representations*.

This process of converting from text to numeric representations is referred to as *embedding* the input to output usable **vectors**, namely **embeddings**. This process is shown in the diagram.

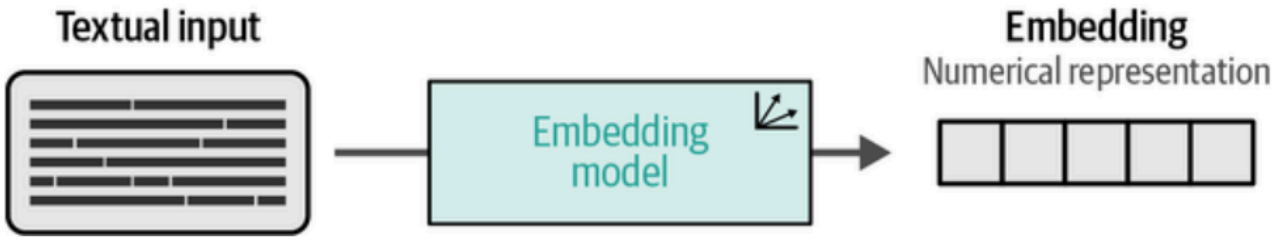

Figure 10-1. We use an embedding model to convert textual input, such as documents, sentences, and phrases, to numerical representations, called embeddings.

This process is performed by an **LLM**, or an **embedding model.**

The purpose of this model is to, accurately as possible, capture the features or *representations* the textual data as an embedding.

To recall, features or representations are the characteristics of the data, which could mean the semantic meaning, patterns, and structure, etc.

But what does it mean to be accurate in representations?

Typically, this means the model can capture the meaning or the *semantic nature* of the text. If the model can capture this, then we would have the right embeddings - important for an embedding space, where *semantic similarity* is calculated using the embedding values.

We've seen this multiple times throughout the book and should have a pretty good understanding at this point. To recall, take a look at the following diagram.

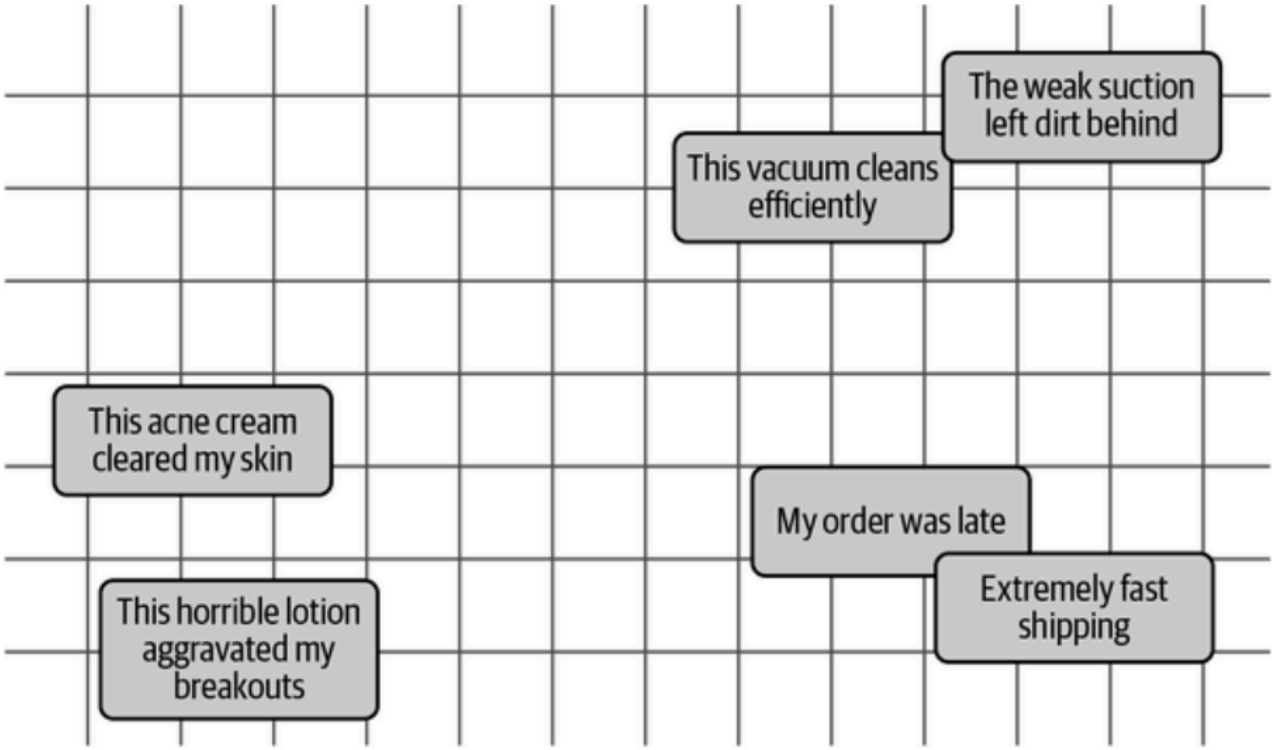

Figure 10-2. The idea of semantic similarity is that we expect textual data with similar meanings to be closer to each other in n-dimensional space (two dimensions are illustrated here).

However, not all embedding models are trained for semantic similarity. For example, when building a sentiment classifier, it is more important to correctly identify the sentiment of the text than it is to correctly generate embeddings based on similarity.

But even for sentiment analysis, similarity (among sentiments) still plays a role.

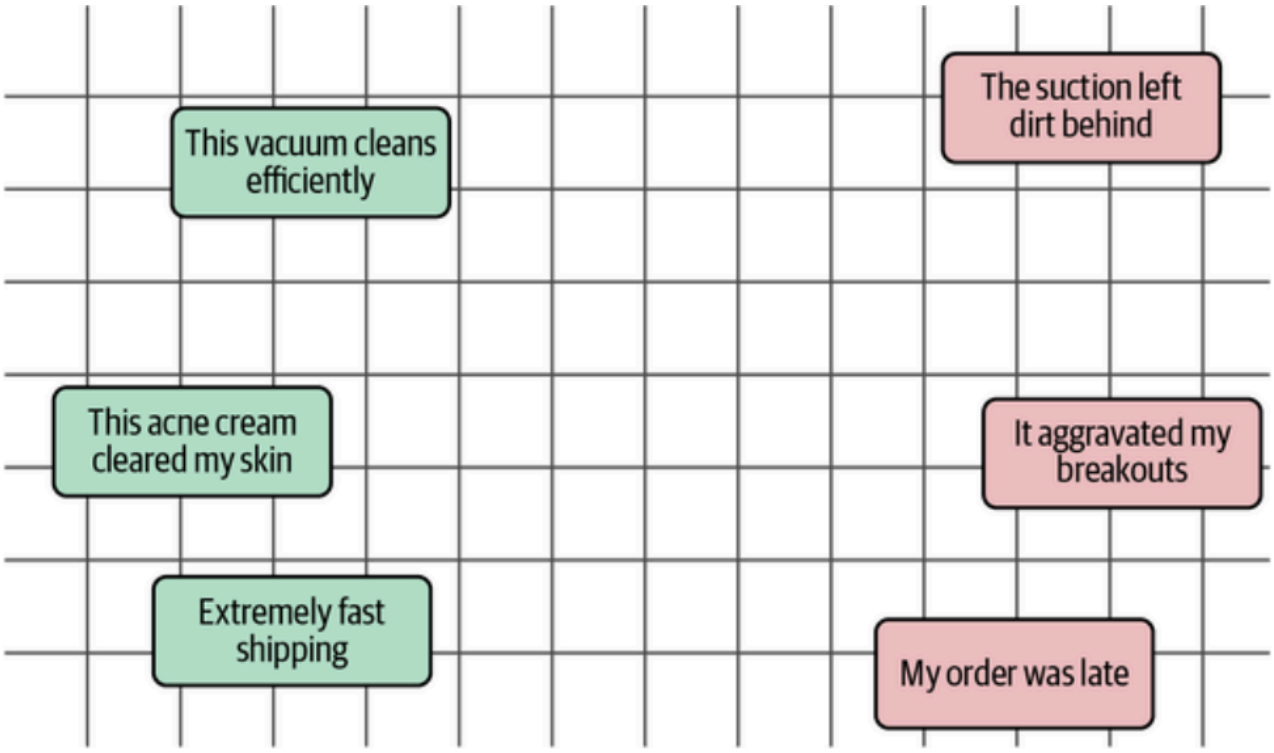

Figure 10-3. In addition to semantic similarity, an embedding model can be trained to focus on sentiment similarity. In this figure, negative reviews (red) are close to one another and dissimilar to positive reviews (green).

To teach or guide a model to capture what's important (based on the use case), there are many ways to train, or fine-tune it.

What Is Contrastive Learning?

Contrastive Learning is a technique that aims to train an embedding model such that **similar documents** are **closer** in vector space while **dissimilar** ones are **further apart**.

The idea of contrastive learning is that the best way to teach a model what are similar and what are not is by giving it dissimilar and similar examples. Hence the name **contrast**ive.

A way to understand contrastive learning better is through explanations.

For example, instead of asking *'Why did you rob the bank?'*, which asks the overall intent of the thief rather than knowing the alternative action (important for classification models), the question could be phrased as *'Why did you rob the bank instead of obeying the law?'*

This technique is called ***contrastive explanation***, where there is **P *instead* of Q**.

Notice that this allows for a more specific 'reason', or in the case of models, differentiation between similar and dissimilar examples.

Another useful example is presented in the diagram:

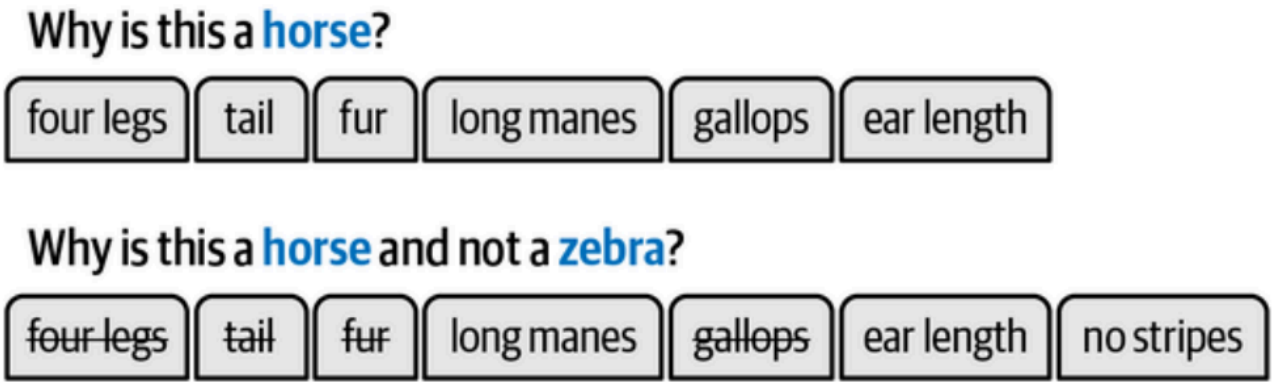

Figure 10-5. When we feed an embedding model different contrasts (degrees of similarity), it starts to learn what makes things different from one another and thereby the distinctive characteristics of concepts.

Notice that getting rid of other characteristics makes the descriptors or *features* more dog-specific.

It's important to know that contrastive learning works **pairwise** or as a **triplet**.

SBERT

`sentence-transformers` is the framework that has popularized **contrastive learning** in the NLP community.

Before `sentence-transformers`, sentence embeddings an architectural structure called **cross-encoders** with BERT.

A cross-encoder allows 2 sentences to be passed to a Transformer at once and directly compares their similarity. While this is powerful and accurate, it uses a lot of computational resources.

For example, when you want to find the highest pair in a collection of 10,000 sentences, it would require n(n−1)/2 = 49,995,000 inference computations.

Also, a cross-encoder does not really generate embeddings, it just generates a similarity score for a `query-document` pair. For similarity search using RAG, for example, we need embeddings of each document and the query, separately to be able to do the search.

A cross-encoder generates embeddings for the **combined** query and document. As you can imagine, using this for similarity search or most use cases at all is not helpful.

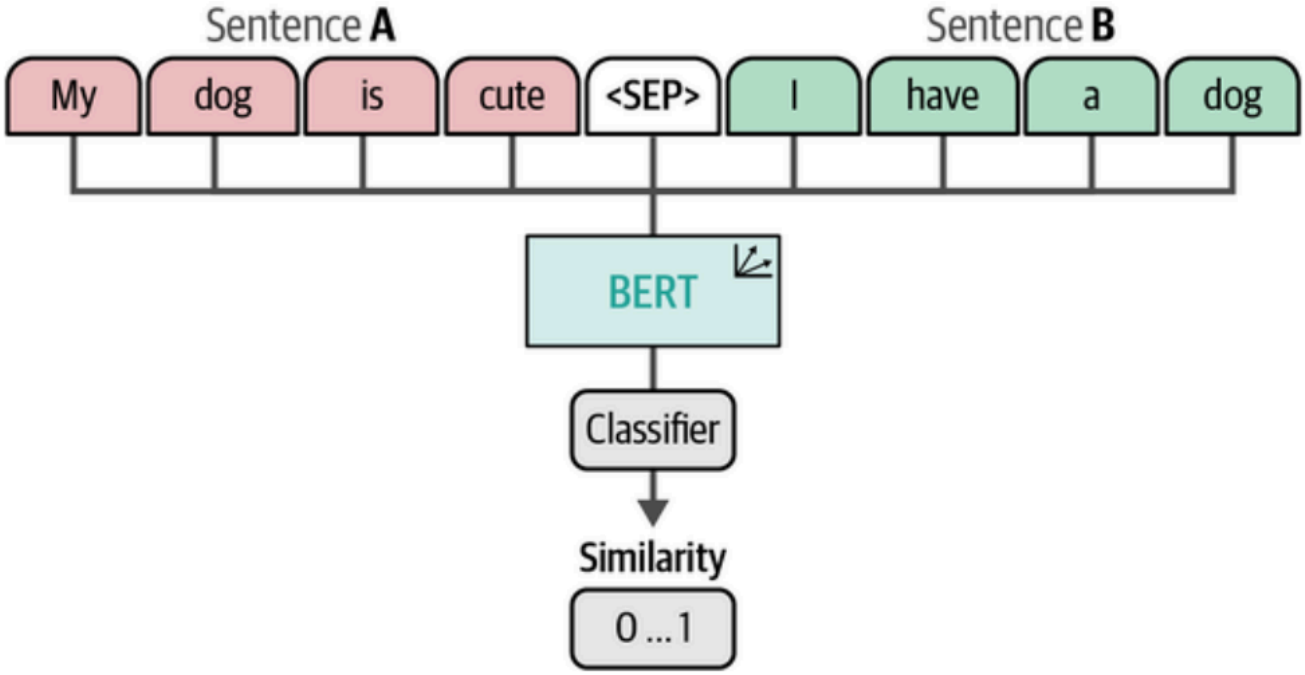

Figure 10-6. The architecture of a cross-encoder. Both sentences are concatenated, separated with a <SEP> token, and fed to the model simultaneously.

It is important to know that cross-encoders are significantly more powerful and accurate. Because of this, they are used as rerankers.

To have a faster alternative, the authors of `sentence-transformers` dropped the classification head (generates similarity score). Instead, bi-encoders output separate embeddings for each sentence individually using mean pooling for the final output layer. The similarity score is calculated using a separate cosine similarity function that takes in a pair of embeddings.

Note: Mean pooling is a design choice, not the only option. The [CLS] token could also be used to extract sentence embeddings, but mean pooling was found to work well empirically for semantic similarity tasks.

Unlike cross-encoders that use a concatenated text embedding, bi-encoders can reuse the generated embeddings for comparison with any other sentence. To give an example, when using a cross-encoder, and there are 100 documents, and assuming you need to find the similarity score for each pair, you would need 100 x 100 feedforward since the embeddings are not saved and reused.

With a bi-encoder, comparing the documents would need 100 embeddings only.

The efficiency comes from **caching** - each document is encoded once and can be compared to any number of queries using fast cosine similarity, versus cross-encoders which must re-encode every query-document pair.

Even with bi-encoders, large-scale systems (like RAG) don't compare against all embeddings naively. Instead, they use **Approximate Nearest Neighbor (ANN)** search with vector

databases (FAISS, Pinecone, Weaviate, ChromaDB, etc.) to organize embeddings spatially and only search relevant regions. This allows systems to search through millions of documents in milliseconds by only comparing against nearby embeddings in the vector space, rather than computing similarity with every single document.

Creating an Embedding Model

Generating Contrastive Examples

Natural Language Inference (NLI) refers to the task of investigating whether, for a given premise, it entails the hypothesis (*entailment*), contradicts it (*contradiction*), or neither (*neutral*).

Contradiction Example:

- **Premise:** '*He is in the cinema watching Coco*'
- **Hypothesis:** '*He is watching Frozen at home*
- They cannot both be true at the same time

Entailment Example

- **Premise:** '*He is in the cinema watching Coco*'
- **Hypothesis:** '*In the movie theater he is watching the Disney movie Coco*'

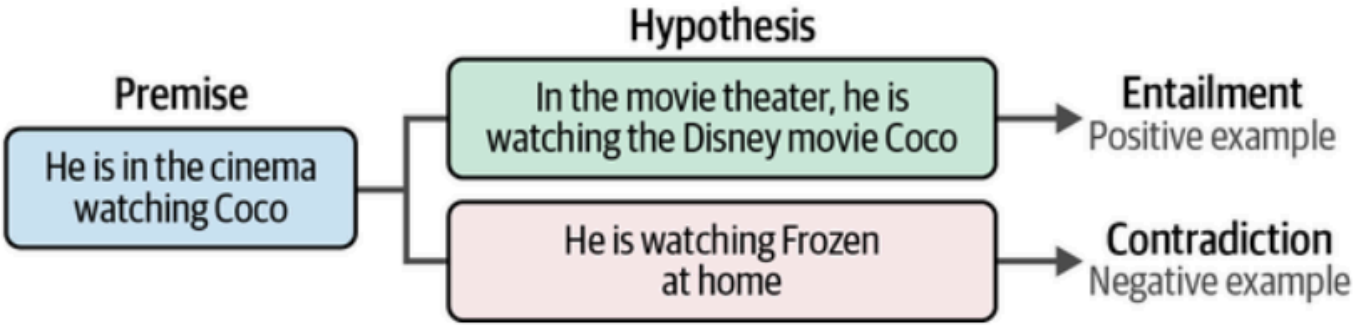

Figure 10-8. We can leverage the structure of NLI datasets to generate negative examples (contradiction) and positive examples (entailments) for contrastive learning.

The data used throughout creating and fine-tuning embedding models is derived from the **General Language Understanding Evaluation** (GLUE) benchmark.

This benchmark consists of nine **language understanding** tasks to evaluate and analyze model performance.

We will be using data for one of the tasks called **Multi-Genre Natural Language Inference (MNLI)** corpus, a collection of 392,702 sentence pairs annotated with entailment (contradiction, neutral, entailment). We will be using 50,000 of these pairs.

```
from datasets import load_dataset
# Load MNLI dataset from GLUE
# 0 = entailment, 1 = neutral, 2 = contradiction
train_dataset = load_dataset("glue", "mnli",
split="train").select(range(50_000))
train_dataset = train_dataset.remove_columns("idx")
```

Let's take a look at an example data:

```
dataset[2]
```

```
{'premise': 'One of our number will carry out your instructions minutely.',
'hypothesis': 'A member of my team will execute your orders with immense
precision.',
'label': 0}
```

Train Model

To train a model from scratch we need to define 2 things:

- A model to embed 2 individual words
- A loss function to optimize the model

We will use BERT base model (uncased):

```
from sentence_transformers import SentenceTransformer
# Use a base model
embedding_model = SentenceTransformer('bert-base-uncased')
```

We will use softmax loss of `sentence-transformers`.

```
from sentence_transformers import losses
# Define the loss function. In softmax loss, we will also need to
explicitly set the number of labels.
train_loss = losses.SoftmaxLoss(
    model=embedding_model,
sentence_embedding_dimension=embedding_model.get_sentence_embedding_dimensio
n(),
num_labels=3
)
```

We also need to define an evaluator to evaluate the model's performance during training, which also determines the best model to save.

This evaluation can be performed using the **Semantic Textual Similarity Benchmark (STSB)** - a collection of human-labeled sentence pairs, with similarity scores between 1 and 5.

We will use this dataset to explore how well the model we're training scores on this semantic similarity task. We also process the STSB data so instead of having a score between 1 and 5, we get it between 0 and 1:

```
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

# Create an embedding similarity evaluator for STSB
val_sts = load_dataset("glue", "stsb", split="validation")
evaluator = EmbeddingSimilarityEvaluator(
    sentences1=val_sts["sentence1"],
    sentences2=val_sts["sentence2"],
    scores=[score/5 for score in val_sts["label"]],
    main_similarity="cosine",
)
```

Then we create, `SentenceTransformerTrainingArguments`, similar to training with **Hugging Face Transformers:**

```
from sentence_transformers.training_args import
SentenceTransformerTrainingArguments

# Define the training arguments
args = SentenceTransformerTrainingArguments(
    output_dir="base_embedding_model",
    num_train_epochs=1,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    warmup_steps=100,
    fp16=True,
    eval_steps=100,
    logging_steps=100,
)
```

Here you can see the following arguments:

- **`num_train_epochs`**
  the number of training rounds. In the example, we keep this at 1, but typically, higher values mean better model performance.

- `per_device_train_batch_size`
  the number of samples to process simultaneously on each device (e.g., GPU or CPU) during evaluation. Higher values generally means faster training.
- `per_device_eval_batch_size`
  the number of samples to process simultaneously on each device (e.g., GPU or CPU) during evaluation. Higher values generally mean faster evaluation.
- `warmup_steps`
  the number of steps during which the learning rate will be linearly increased from zero to the initial learning rate defined for the training process. Note that we did not specify a custom learning rate for this training process.
- `fp16`
  by enabling this, we allow for mixed precision training, where computations are performed using 16-bit floating-point numbers (fp16) instead of the default 32-bit (fp32). This reduces memory usage and potentially increases the training speed.

Now that we have the model (**BERT base model (uncased)**), loss (**softmax**), and evaluator (**SBST**), we can start with the training:

```
from sentence_transformers.trainer import SentenceTransformerTrainer

# Train embedding model
trainer = SentenceTransformerTrainer(
    model = embedding_model,
    args = args,
    train_dataset = train_dataset,
    loss = train_loss,
    evaluator = evaluator
)

trainer.train()
```

After training the model, we can use the evaluator to get the performance on this single task:

```
# Evaluate our trained model
evaluator(embedding_model)
```

```
{'pearson_cosine': 0.5982288436666162,
'spearman_cosine': 0.6026682018489217,
'pearson_manhattan': 0.6100690915500567,
'spearman_manhattan': 0.617732600131989,
'pearson_euclidean': 0.6079280934202278,
'spearman_euclidean': 0.6158926913905742,
```

```
'pearson_dot': 0.38364924527804595,
'spearman_dot': 0.37008497926991796,
'pearson_max': 0.6100690915500567,
'spearman_max': 0.617732600131989}
```

We are interested in `pearson_cosine` - the cosine similarity between centered vectors.

In-Depth Evaluation

In addition to the previously mentioned **GLUE** benchmark, there exist many more benchmarks that allow for the evaluation of embedding models.

To unify these evaluations, the **Massive Text Embedding Benchmark (MTEB)** was developed. It spans 8 embedding tasks and cover 58 datasets and 112 evaluations. An example is provided below:

```
from mteb import MTEB

# Choose evaluation task
evaluation = MTEB(tasks=["Banking77Classification"])

# Calculate results
results = evaluation.run(model)
```

```
{'Banking77Classification': {'mteb_version': '1.1.2',
'dataset_revision': '0fd18e25b25c072e09e0d92ab615fda904d66300',
'mteb_dataset_name': 'Banking77Classification',
'test': {'accuracy': 0.4926298701298701,
'f1': 0.49083335791288685,
'accuracy_stderr': 0.010217785746224237,
'f1_stderr': 0.010265814957074591,
'main_score': 0.4926298701298701,
'evaluation_time': 31.83}}}
```

Loss Functions

We trained our model using softmax loss to illustrate how one of the first `sentence-transformers` models was trained. But *softmax* is generally not advised, as there are better loss functions.

Two loss functions perform well generally:

1. **Cosine Similarity**
2. **Multiple Negatives Ranking (MNR) Loss**

Cosine Similarity

Typically used in semantic textual similarity tasks. In these tasks, a similarity score is assigned for a pair of texts to optimize the model.

Instead of having classification (e.g., similar/dissimilar, positive/negative), cosine similarity quantifies the degree to which 2 texts are similar or not.

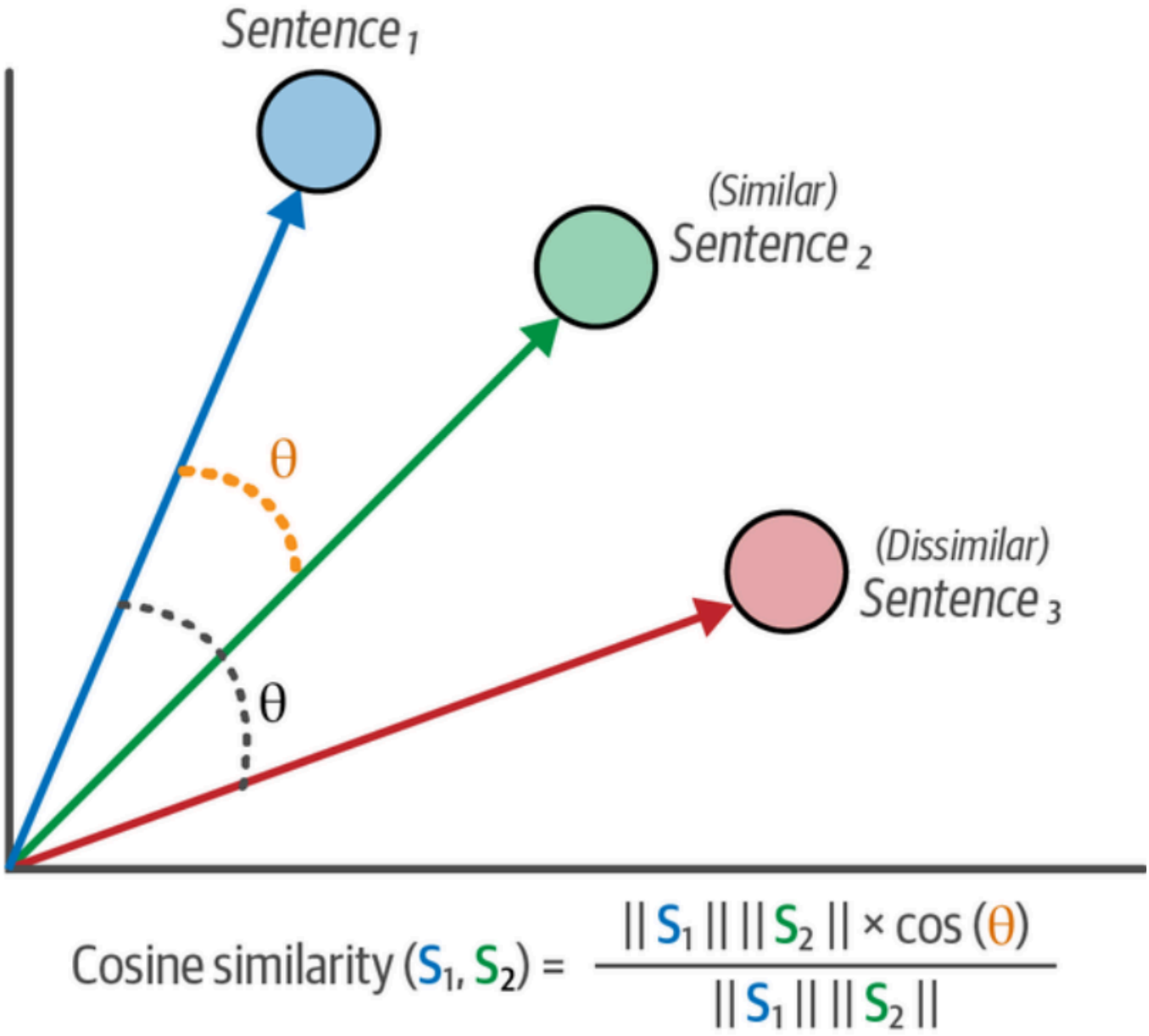

Figure 10-9. Cosine similarity loss aims to minimize the cosine distance between semantically similar sentences and to maximize the distance between semantically dissimilar sentences.

Cosine similarity loss is straightforward: after calculating the cosine similarity score for 2 text embeddings, this score is compared to the labeled similarity score of the evaluator.

Cosine similarity is usually a value between 0 and 1. To use the loss with the NLI, we need to convert the values **entailment (0)**,

neutral (1), and **contradiction (2)** to values between **0 and 1.**

We know that **entailment** means **more similar**, and **contradiction and neutral** mean **dissimilar.**

```
from datasets import Dataset, load_dataset

# Load MNLI dataset from GLUE
# 0 = entailment, 1 = neutral, 2 = contradiction
train_dataset = load_dataset(
    "glue", "mnli", split="train"
).select(range(50_000))
train_dataset = train_dataset.remove_columns("idx")

# (neutral/contradiction)=0 and (entailment)=1
mapping = {2: 0, 1: 0, 0:1}
train_dataset = Dataset.from_dict({
    "sentence1": train_dataset["premise"],
    "sentence2": train_dataset["hypothesis"],
    "label": [float(mapping[label]) for label in train_dataset["label"]]
})
```

Let's create our evaluator:

```
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

# Create an embedding similarity evaluator for stsb
val_sts = load_dataset("glue", "stsb", split="validation")
evaluator = EmbeddingSimilarityEvaluator(
    sentences1=val_sts["sentence1"],
    sentences2=val_sts["sentence2"],
    scores=[score/5 for score in val_sts["label"]],
    main_similarity="cosine"
)
```

Then, we follow the same steps as before to train the model but select a **different loss** (**cosine similarity loss**) instead:

```
from sentence_transformers import losses, SentenceTransformer
from sentence_transformers.trainer import SentenceTransformerTrainer
from sentence_transformers.training_args import
SentenceTransformerTrainingArguments

# Define model
```

```
embedding_model = SentenceTransformer("bert-base-uncased")

# Loss function
train_loss = losses.CosineSimilarityLoss(model=embedding_model)

# Define the training arguments
args = SentenceTransformerTrainingArguments(
    output_dir="cosineloss_embedding_model",
    num_train_epochs=1,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    warmup_steps=100,
    fp16=True,
    eval_steps=100,
    logging_steps=100,
)

# Train model
trainer = SentenceTransformerTrainer(
    model=embedding_model,
    args=args,
    train_dataset=train_dataset,
    loss=train_loss,
    evaluator=evaluator
)
trainer.train()
```

After the training, we can evaluate the performance of the model:

```
# Evaluate our trained model
evaluator(embedding_model)
```

```
{'pearson_cosine': 0.7222322163831805,
'spearman_cosine': 0.7250508271229599,
'pearson_manhattan': 0.7338163436711481,
'spearman_manhattan': 0.7323479193408869,
'pearson_euclidean': 0.7332716434966307,
'spearman_euclidean': 0.7316999722750905,
'pearson_dot': 0.660366792336156,
'spearman_dot': 0.6624167554844425,'pearson_max': 0.7338163436711481,
'spearman_max': 0.7323479193408869}
```

Remember that the pearson cosine was **0.59**. This increase to **0.72** shows the importance and impact of loss function on performance.

Multiple Negatives Ranking (MNR) Loss

MNR Loss, also referred to as **InfoNCE** or **NTXentLoss**, uses either **positive pairs** of sentences or triplets that contain a pair of positive sentences and an additional unrelated sentence (called a **negative**).

This negative sentence **represents the dissimilarity between the positive sentences.**

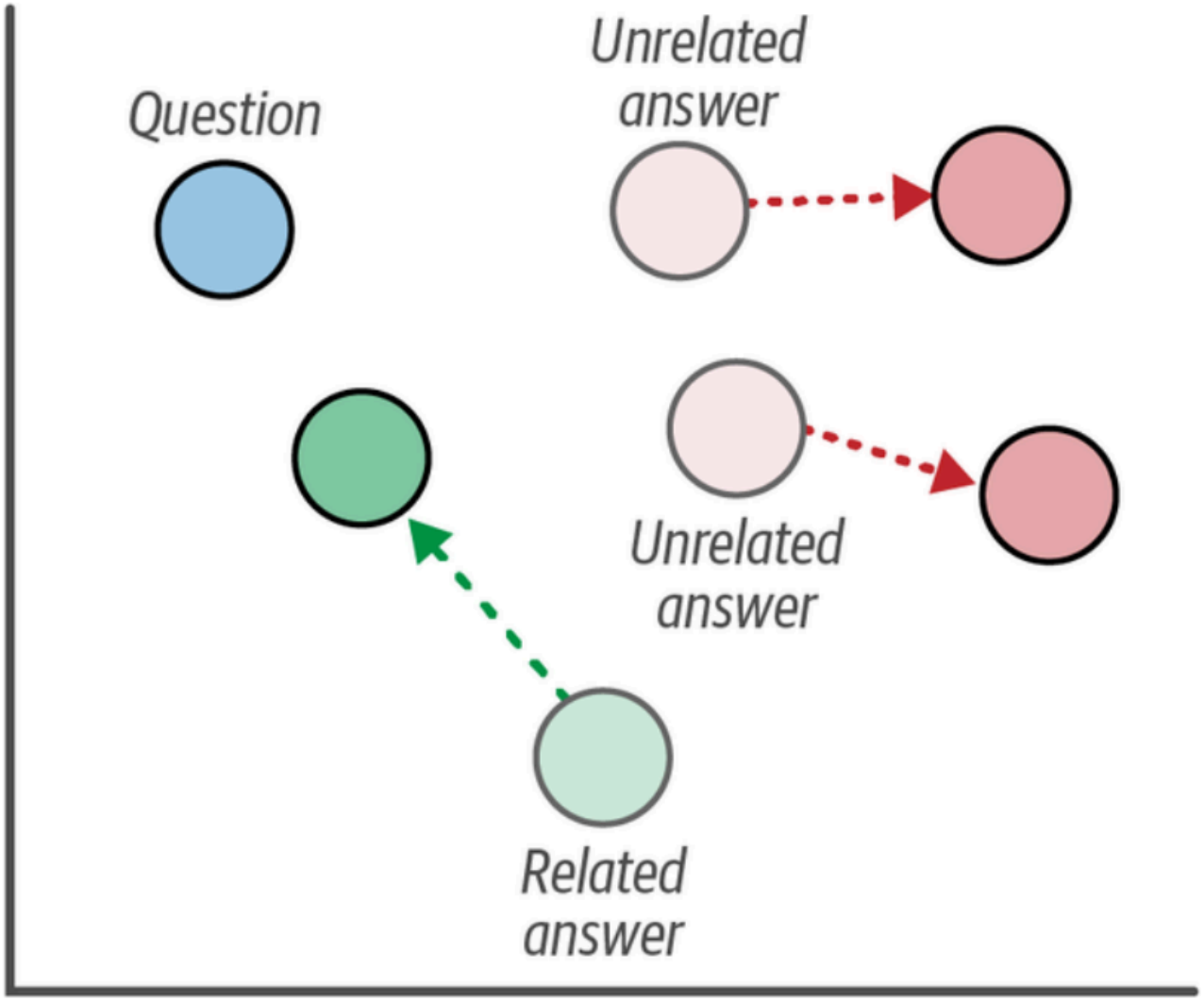

Figure 10-10. Multiple negatives ranking loss aims to minimize the distance between related pairs of text, such as questions and answers, and maximize the distance between unrelated pairs, such as questions and unrelated answers.

Example:

Pairs:

- Apple - Banana (fruit - fruit)
- Tiger - Cat (animal - animal)
- Chopper - Airplane (Flying vehicles)

Positive pair:

- Apple (anchor sentence) - Banana (positive)

Negative pair:

- Apple - Airplane (negative)
- Apple - Cat (negative)

After we have these negative pairs, we generate their embeddings and calculate their cosine similarity.

The similarity score is used to know whether they are similar or not (works like classification). Then, cross-entropy loss is used to optimize the model.

To make these pairs from the MNLI dataset that we have, we start with anchor sentence (i.e., labeled as 'premise'), then we select this anchor's positive sentence by only selecting those that are labeled 'entailment'. To add negative sentences, we randomly sample sentences as the 'hypothesis.'

```
import random
from tqdm import tqdm
from datasets import Dataset, load_dataset

# # Load MNLI dataset from GLUE
mnli = load_dataset("glue", "mnli",
split="train").select(range(50_000))
mnli = mnli.remove_columns("idx")
mnli = mnli.filter(lambda x: True if x["label"] == 0 else False)

# Prepare data and add a soft negative
train_dataset = {"anchor": [], "positive": [], "negative": []}
soft_negatives = mnli["hypothesis"]
random.shuffle(soft_negatives)
for row, soft_negative in tqdm(zip(mnli, soft_negatives)):
    train_dataset["anchor"].append(row["premise"])
    train_dataset["positive"].append(row["hypothesis"])
```

```
    train_dataset["negative"].append(soft_negative)
train_dataset = Dataset.from_dict(train_dataset)
```

After selecting only 'entailment', we reduced the number of rows from the selected 50,000 to 16,875.

Let's define the evaluator:

```
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

# Create an embedding similarity evaluator for stsb
val_sts = load_dataset("glue", "stsb", split="validation")
evaluator = EmbeddingSimilarityEvaluator(
    sentences1=val_sts["sentence1"],
    sentences2=val_sts["sentence2"],
    scores=[score/5 for score in val_sts["label"]],
    main_similarity="cosine"
)
```

We use the same code as before but we use MNR loss instead of cosine similarity loss:

```
from sentence_transformers import losses, SentenceTransformer
from sentence_transformers.trainer import SentenceTransformerTrainer
from sentence_transformers.training_args import
SentenceTransformerTrainingArguments

# Define model
embedding_model = SentenceTransformer('bert-base-uncased')

# Loss function
train_loss = losses.MultipleNegativesRankingLoss(model=embedding_model)

# Define the training arguments
args = SentenceTransformerTrainingArguments(
    output_dir="mnrloss_embedding_model",
    num_train_epochs=1,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    warmup_steps=100,
    fp16=True,
    eval_steps=100,
    logging_steps=100,
)

# Train model
```

```
trainer = SentenceTransformerTrainer(
    model=embedding_model,
    args=args,
    train_dataset=train_dataset,
    loss=train_loss,
    evaluator=evaluator
)
trainer.train()
```

Let's see how MNR loss compares to previous loss functions:

```
# Evaluate our trained model
evaluator(embedding_model)
```

```
{'pearson_cosine': 0.8093892326162132,
'spearman_cosine': 0.8121064796503025,
'pearson_manhattan': 0.8215001523827565,
'spearman_manhattan': 0.8172161486524246,
'pearson_euclidean': 0.8210391407846718,
'spearman_euclidean': 0.8166537141010816,
'pearson_dot': 0.7473360302629125,
'spearman_dot': 0.7345184137194012,
'pearson_max': 0.8215001523827565,
'spearman_max': 0.8172161486524246}
```

Recall that the pearson cosine from when we used softmax was **0.59**, with cosine **0.72**, and now we have **0.80**.

Note that since we sampled from other question/answer pairs, the negatives that we used are potentially completely different and unrelated to the question (anchor). This makes it really easy for the model to differentiate them. In real life, this would mean that the model would struggle to differentiate between sentences such as: '***How many people live in Amsterdam?***' vs '***More than a million people live in Utrecht, which is more than Amsterdam***'

To address this, we would need to have negatives that are very related to the answer but is not the right answer, like the example sentence given above. These negatives are called **hard negatives**

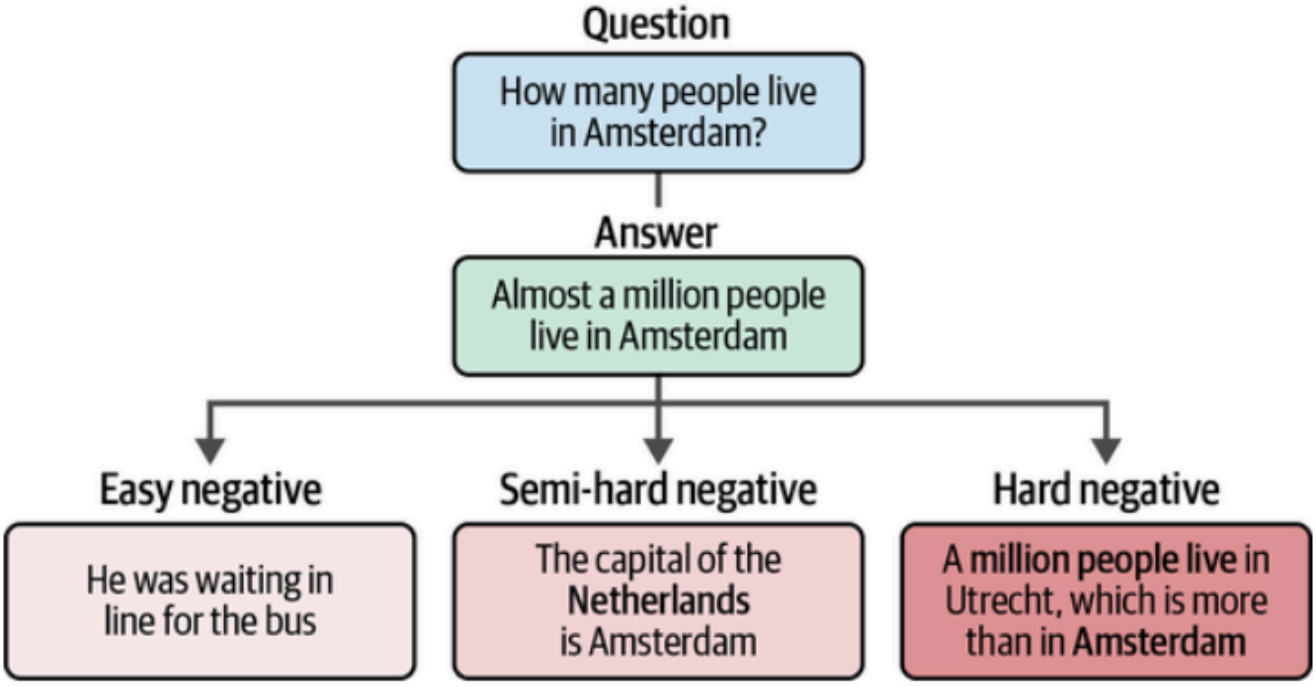

Figure 10-11. An easy negative is typically unrelated to both the question and answer. A semi-hard negative has some similarities to the topic of the question and answer but is somewhat unrelated. A hard negative is very similar to the question but is generally the wrong answer.

Gathering negatives can be divided into three processes:

1. ***Easy Negatives***
   Through random sampling documents (what was done in the example).
2. ***Semi-hard Negatives***
   Using an embedding model to generate embeddings for sentences and using cosine similarity to find those that are highly related. This does not lead to hard negatives since it merely finds similar sentences, not Q/A pairs.
3. ***Hard Negatives***
   These are manually labeled or a generative model is used to either label or generate sentence pairs.

Fine-Tuning an Embedding Model

Training a model from scratch can be time-consuming and resource-intensive. A more efficient method to teach a model to perform specific tasks is to **fine-tune** it.

Supervised

This is similar to how we trained the model in the previous sections. Recall that supervised just means we show the model labels to data points.

For this, we use `all-MiniLM-L6-v2` from `sentence-transformers`

We use the same data (MNLI) we used for MNR Loss for the evaluator:

```
from datasets import load_dataset
from sentence_transformers.evaluation
import EmbeddingSimilarityEvaluator

# Load MNLI dataset from GLUE
# 0 = entailment, 1 = neutral, 2 = contradiction
train_dataset = load_dataset(
    "glue", "mnli", split="train"
).select(range(50_000))
train_dataset = train_dataset.remove_columns("idx")

# Create an embedding similarity evaluator for stsb
val_sts = load_dataset("glue", "stsb", split="validation")
evaluator = EmbeddingSimilarityEvaluator(
    sentences1=val_sts["sentence1"],
    sentences2=val_sts["sentence2"],
    scores=[score/5 for score in val_sts["label"]],
    main_similarity="cosine"
)
```

Then we replace `bert-base-uncased` with the pretrained model:

```
from sentence_transformers import losses, SentenceTransformer
from sentence_transformers.trainer import SentenceTransformerTrainer
from sentence_transformers.training_args import
SentenceTransformerTrainingArguments

# Define model
embedding_model = SentenceTransformer('sentence-transformers/all-MiniLM-L6-
v2')

# Loss function
train_loss = losses.MultipleNegativesRankingLoss(model=embedding_model)

# Define the training arguments
args = SentenceTransformerTrainingArguments(
    output_dir="finetuned_embedding_model",
    num_train_epochs=1,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    warmup_steps=100,
    fp16=True,
    eval_steps=100,
    logging_steps=100,
)
```

```
# Train model
trainer = SentenceTransformerTrainer(
    model=embedding_model,
    args=args,
    train_dataset=train_dataset,
    loss=train_loss,
    evaluator=evaluator
)
trainer.train()
```

Evaluating this model gives us the following score:

```
# Evaluate our trained model
evaluator(embedding_model)
```

```
{'pearson_cosine': 0.8509553350510896,
'spearman_cosine': 0.8484676559567688,
'pearson_manhattan': 0.8503896832470704,
'spearman_manhattan': 0.8475760325664419,
'pearson_euclidean': 0.8513115442079158,
'spearman_euclidean': 0.8484676559567688,
'pearson_dot': 0.8489553386816947,
'spearman_dot': 0.8484676559567688,
'pearson_max': 0.8513115442079158,
'spearman_max': 0.8484676559567688}
```

Although a score of 0.85 is the highest we have seen thus far, the pretrained model that we used for fine-tuning was already trained on the full MNLI dataset, whereas we only used 50,000 examples.

Augmented SBERT

A disadvantage of training or fine-tuning embedding models is that they require substantial training data.

One way to augment data such that an embedding model can be fine-tuned when there is only a little **labeled** data available is called ***Augmented SBERT***.

Augmented SBERT involves the following steps:

1. Fine-tune a cross-encoder (BERT) using a small, annotated dataset (**gold dataset**)
2. Create new sentence pairs (or collect, unlabeled)

3. Use the fine-tuned cross encoder to label the unlabeled sentence pairs (results to **silver dataset**)
4. Train a bi-encoder (SBERT) on the extended dataset (**gold + silver**)

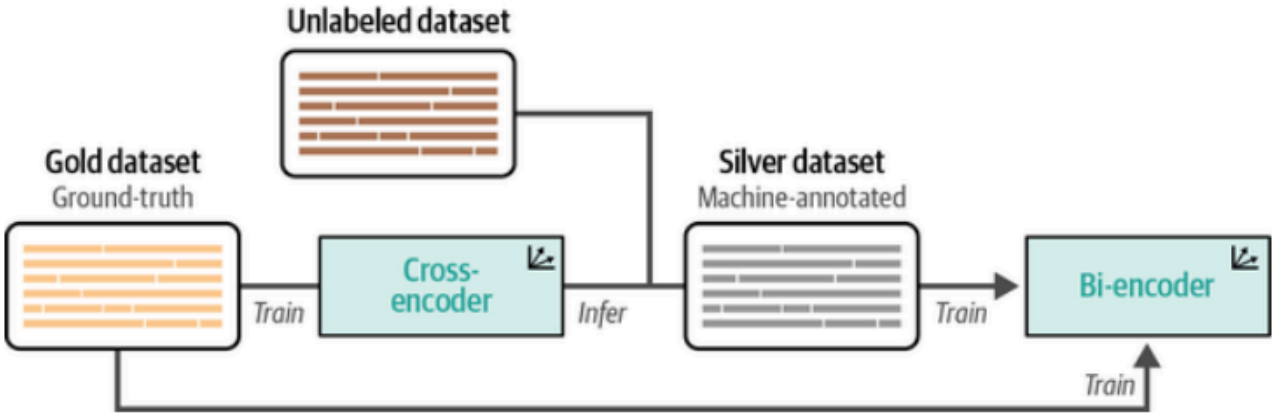

Figure 10-12. Augmented SBERT works through training a cross-encoder on a small gold dataset, then using that to label an unlabeled dataset to generate a larger silver dataset. Finally, both the gold and silver datasets are used to train the bi-encoder.

To simulate a situation where there is little labeled data available, let's use 10,000 instead of 50,000:

```
import pandas as pd
from tqdm import tqdm
from datasets import load_dataset, Dataset
from sentence_transformers import InputExample
from sentence_transformers.datasets import NoDuplicatesDataLoader

# Prepare a small set of 10000 documents for the cross-encoder
dataset = load_dataset("glue", "mnli",
split="train").select(range(10_000))
mapping = {2: 0, 1: 0, 0:1}

# Data loader
gold_examples = [
    InputExample(texts=[row["premise"], row["hypothesis"]],
label=mapping[row["label"]])
    for row in tqdm(dataset)
]
gold_dataloader = NoDuplicatesDataLoader(gold_examples, batch_size=32)

# Pandas DataFrame for easier data handling
gold = pd.DataFrame(
    {
    "sentence1": dataset["premise"],
    "sentence2": dataset["hypothesis"],
    "label": [mapping[label] for label in dataset["label"]]
```

```
    }
)
```

The result is the **gold dataset** and it represents our **ground truth**.

We train the cross-encoder with this gold dataset (**Step 1**):

```
from sentence_transformers.cross_encoder import CrossEncoder

# Train a cross-encoder on the gold dataset
cross_encoder = CrossEncoder("bert-base-uncased", num_labels=2)
cross_encoder.fit(
    train_dataloader=gold_dataloader,
    epochs=1,
    show_progress_bar=True,
    warmup_steps=100,
    use_amp=False
)
```

Then we use the remaining 40,000 (from the 50,000) sentence pairs (**silver dataset**) as our silver dataset (**Step 2**):

```
# Prepare the silver dataset by predicting labels with the cross-encoder
silver = load_dataset(
    "glue", "mnli", split="train"
).select(range(10_000, 50_000))
pairs = list(zip(silver["premise"], silver["hypothesis"]))
```

Then, we label these 40,000 unlabeled sentence pairs using the **fine-tuned cross-encoder** (**Step 3**):

```
import numpy as np

# Label the sentence pairs using our fine-tuned cross-encoder
output = cross_encoder.predict(
    pairs, apply_softmax=True,
    show_progress_bar=True
)

silver = pd.DataFrame(
    {
        "sentence1": silver["premise"],
        "sentence2": silver["hypothesis"],
```

```
        "label": np.argmax(output, axis=1)
    }
)
```

This output of this is the **silver dataset** which we combine with the gold dataset.

We then use this extended dataset (**gold + silver**) to train `bert-base-uncased` as bi-encoder (**Step 4**):

```
# Combine gold + silver
data = pd.concat([gold, silver], ignore_index=True, axis=0)
data = data.drop_duplicates(subset=["sentence1", "sentence2"], keep="first")
train_dataset = Dataset.from_pandas(data, preserve_index=False)
```

Let's define the evaluator:

```
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

# Create an embedding similarity evaluator for stsb
val_sts = load_dataset("glue", "stsb", split="validation")
evaluator = EmbeddingSimilarityEvaluator(
    sentences1=val_sts["sentence1"],
    sentences2=val_sts["sentence2"],
    scores=[score/5 for score in val_sts["label"]],
    main_similarity="cosine"
)
```

Now we train the bi-encoder on the augmented dataset:

```
from sentence_transformers import losses, SentenceTransformer
from sentence_transformers.trainer import SentenceTransformerTrainer
from sentence_transformers.training_args import
SentenceTransformerTrainingArguments

# Define model
embedding_model = SentenceTransformer("bert-base-uncased")

# Loss function
train_loss = losses.CosineSimilarityLoss(model=embedding_model)

# Define the training arguments
args = SentenceTransformerTrainingArguments(
    output_dir="augmented_embedding_model",
```

```
    num_train_epochs=1,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    warmup_steps=100,
    fp16=True,
    eval_steps=100,
    logging_steps=100,
)

# Train model
trainer = SentenceTransformerTrainer(
    model=embedding_model,
    args=args,
    train_dataset=train_dataset,
    loss=train_loss,
    evaluator=evaluator
)
trainer.train()
```

Finally, we evaluate the model:

```
evaluator(embedding_model)
```

```
{'pearson_cosine': 0.7101597020018693,
'spearman_cosine': 0.7210536464320728,
'pearson_manhattan': 0.7296749443525249,
'spearman_manhattan': 0.7284184255293913,
'pearson_euclidean': 0.7293097297208753,
'spearman_euclidean': 0.7282830906742256,
'pearson_dot': 0.6746605824703588,
'spearman_dot': 0.6754486790570754,
'pearson_max': 0.7296749443525249,
'spearman_max': 0.7284184255293913}
```

The original cosine similarity loss example had a score of **0.72** with the full dataset. Using only **20%** of that data, we managed to get a score of **0.71**. This shows the importance of Augmented SBERT when limited labeled data.

Unsupervised Learning

Not all real-world and publicly available data are labeled, and labeling them could be time-consuming and expensive.

Unsupervised learning is a technique to train models when predetermined labels are not available.

Transformer-Based Sequential Denoising Auto-Encoder

The idea of **TSDAE** is that we add noise to the input sentence by removing a certain percentage of words from it. A sentence embedding is then generated from this '*damaged*' sentence.

A decoder then tries to reconstruct this sentence but without the artificial noise. The performance of the embedding model (encoder) is calculated based on how well the decoder reconstructed the sentence.

How is reconstruction evaluated?

The reconstruction is evaluated using **cross-entropy loss** at the token level:

1. The decoder generates the reconstructed sentence **token by token** (word by word)
2. At each position, the decoder outputs a **probability distribution over the entire vocabulary** - predicting what word should come next
3. The loss compares this predicted probability distribution to the **actual token** that appears in the original sentence
4. **Cross-entropy loss** measures how far off the prediction was from the correct token

Example:

- Original sentence: "*The capital of the Netherlands is Amsterdam*"
- Damaged sentence: "*.... capital of ..... is Amsterdam*"
- Decoder tries to reconstruct: "*The capital of the Netherlands is Amsterdam*"
  - Position 1: Predicts "*The*" → compares to actual "*The*" → calculate loss
  - Position 2: Predicts "*capital*" → compares to actual "*capital*" → calculate loss
  - Position 4: Predicts "*the*" → compares to actual "*the*" → calculate loss
  - etc.

The total loss is summed across all token positions. The lower the loss, the better the reconstruction, which means the encoder created a good embedding that captured the semantic meaning even from the damaged input.

This method is very similar to masked language modeling, where we try to reconstruct and learn certain masked words. Here, instead of reconstructing masked words, we try to reconstruct the entire sentence.

After training, we can use the encoder to generate embeddings from text since the decoder is only used for judging whether the embeddings can accurately reconstruct the original sentence.

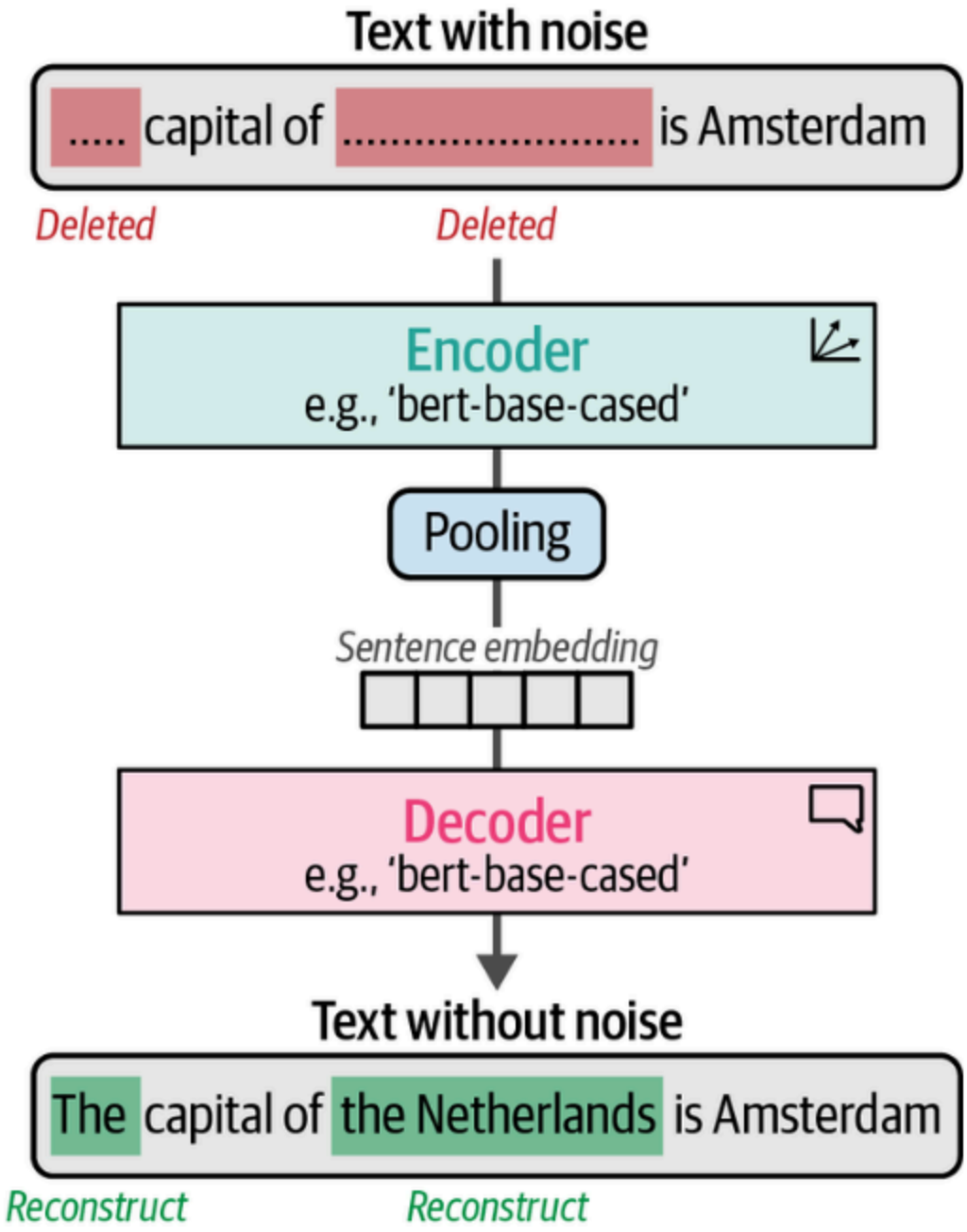

Figure 10-13. TSDAE randomly removes words from an input sentence that is passed through an encoder to generate a sentence embedding. From this sentence embedding, the original sentence is reconstructed.

Since we only need a bunch of sentences without any labels, training this model is straightforward. We start by downloading an external tokenizer, which is used for the denoising procedure:

```
# Download additional tokenizer
import nltk
nltk.download("punkt")
```

Then, we create flat sentences from our data and remove any labels to mimic an unsupervised setting:

```
from tqdm import tqdm
from datasets import Dataset, load_dataset
from sentence_transformers.datasets import DenoisingAutoEncoderDataset

# Create a flat list of sentences
mnli = load_dataset("glue", "mnli",
split="train").select(range(25_000))
flat_sentences = mnli["premise"] + mnli["hypothesis"]

# Add noise to our input data
damaged_data = DenoisingAutoEncoderDataset(list(set(flat_sentences)))

# Create dataset
train_dataset = {"damaged_sentence": [], "original_sentence": []}
for data in tqdm(damaged_data):
    train_dataset["damaged_sentence"].append(data.texts[0])
    train_dataset["original_sentence"].append(data.texts[1])
train_dataset = Dataset.from_dict(train_dataset)
```

This creates a dataset of 50,000 sentences. Let's take a look at an example:

```
train_dataset[0]
```

```
{'damaged_sentence': 'Grim jaws are.',
'original_sentence': 'Grim faces and hardened jaws are not people-
friendly.'}
```

The first sentence shows the "noisy" data whereas the second shows the original input sentence.

Let's define the evaluator:

```
from sentence_transformers.evaluation import EmbeddingSimilarityEvaluator

# Create an embedding similarity evaluator for stsb
val_sts = load_dataset("glue", "stsb", split="validation")
evaluator = EmbeddingSimilarityEvaluator(
    sentences1=val_sts["sentence1"],
    sentences2=val_sts["sentence2"],
    scores=[score/5 for score in val_sts["label"]],
```

```
    main_similarity="cosine"
)
```

We run the training as before but with the [CLS] token as the pooling strategy instead of mean pooling of the token embeddings. In the TSDAE paper, this was shown to be more effective since mean pooling loses the position information, which is not the case when using the [CLS] token:

```
from sentence_transformers import models, SentenceTransformer

# Create your embedding model
word_embedding_model = models.Transformer("bert-base-uncased")
pooling_model = models.Pooling(
    word_embedding_model.get_word_embedding_dimension(), "cls"
)
embedding_model = SentenceTransformer(
    modules=[word_embedding_model, pooling_model]
)
```

We use the **DenoisingAutoEncoderLoss** loss function that attempts to reconstruct the original sentence using the noisy sentence. By doing so, it will learn how to accurately represent the data. It is similar to masking but without knowing where the actual masks are.

We also tie the parameters of both models. Instead of having separate weights for the encoder's embedding layer and the decoder's output layer, they share the same weights. This means that any updates to the weights in one layer will be reflected in the other layer as well:

```
from sentence_transformers import losses

# Use the denoising auto-encoder loss
train_loss = losses.DenoisingAutoEncoderLoss(
    embedding_model, tie_encoder_decoder=True
)
train_loss.decoder = train_loss.decoder.to("cuda")
```

Training our model works the same as we have seen several times before but we lower the batch size as memory increases with this loss function:

```
from sentence_transformers.trainer import SentenceTransformerTrainer
from sentence_transformers.training_args import
SentenceTransformerTrainingArguments

# Define the training arguments
```

```
args = SentenceTransformerTrainingArguments(
    output_dir="tsdae_embedding_model",
    num_train_epochs=1,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    warmup_steps=100,
    fp16=True,
    eval_steps=100,
    logging_steps=100,
)

# Train model
trainer = SentenceTransformerTrainer(
    model=embedding_model,
    args=args,
    train_dataset=train_dataset,
    loss=train_loss,
    evaluator=evaluator
)
trainer.train()
```

Let's evaluate the model:

```
# Evaluate our trained model
evaluator(embedding_model)
```

```
{'pearson_cosine': 0.6991809700971775,
'spearman_cosine': 0.713693213167873,
'pearson_manhattan': 0.7152343356643568,
'spearman_manhattan': 0.7201441944880915,
'pearson_euclidean': 0.7151142243297436,
'spearman_euclidean': 0.7202291660769805,
'pearson_dot': 0.5198066451871277,
'spearman_dot': 0.5104025515225046,
'pearson_max': 0.7152343356643568,
'spearman_max': 0.7202291660769805}
```

We get a score of 0.70 using unlabeled data.

Using TSDAE for Domain Adaptation

When you have very little or no labeled data available, you typically use unsupervised learning to create your text embedding model. However, unsupervised techniques are generally outperformed by supervised techniques and have difficulty learning domain-specific concepts.

This is where **domain adaptation** comes in. Its goal is to update existing embedding models to a specific textual domain that contains different subjects from the source domain.

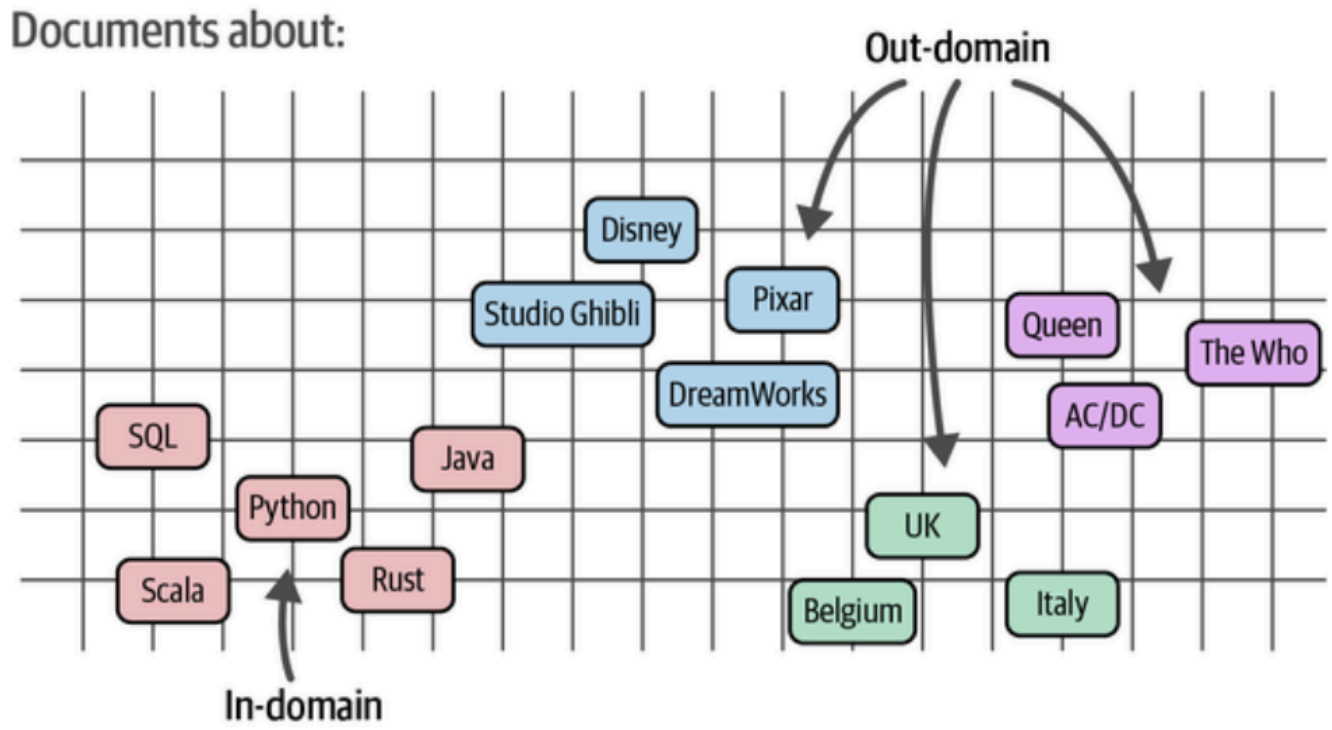

Figure 10-14. In domain adaptation, the aim is to create and generalize an embedding model from one domain to another.

The **target domain**, or **out-domain**, generally contains words and subjects that were not found in the **source domain** or **in-domain**.

One method for domain adaptation is called **adaptive pretraining**. You start by pretraining your domain-specific corpus using an unsupervised technique, such as the previously discussed TSDAE or masked language modeling. Then, you fine-tune that model using a training dataset that can be either outside or in your target domain.

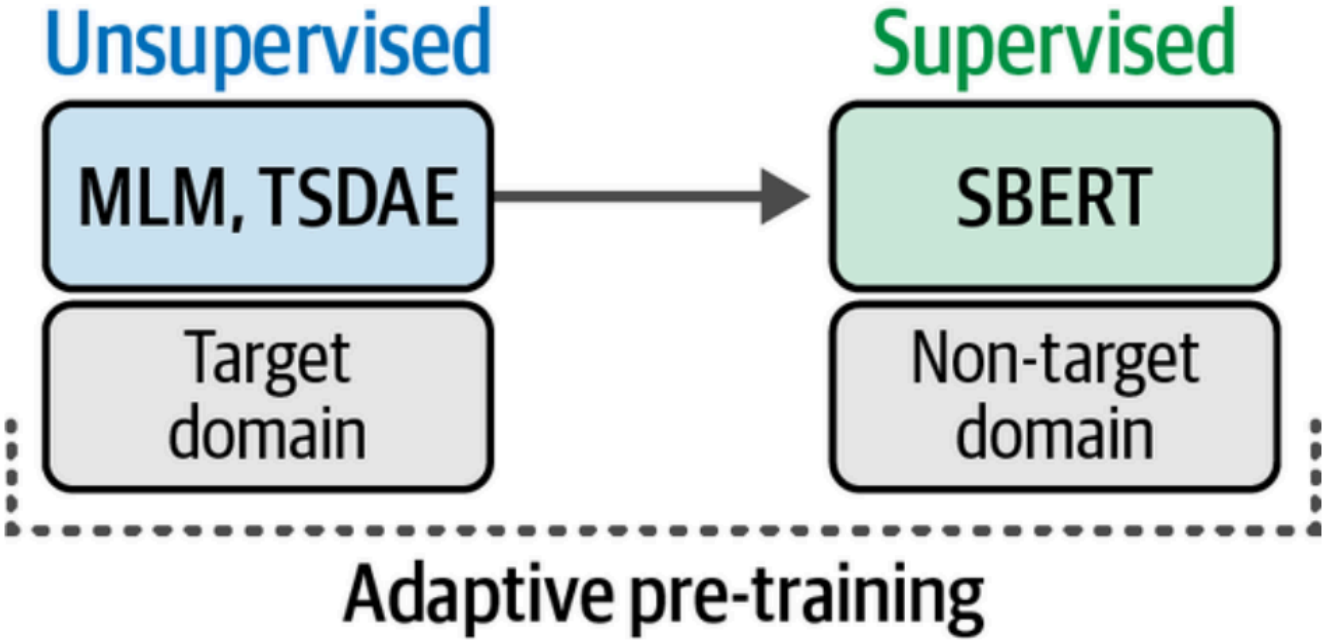

Figure 10-15. Domain adaptation can be performed with adaptive pretraining and adaptive fine-tuning.

Although data from the target domain is preferred, out-domain data also works since we started with unsupervised training on the target domain.